CGT 581 | Assignment 1 | Procedural Seating Furniture Generator Report

Angshuman Mazumdar | Link To Video

This assignment covered a project that could procedurally generate a piece of seating furniture (chair, sofa, ottoman, bench etc.).

The main idea was set up first as a hierarchical structure, which can be represented as following:

- Base Grid Blocks
  - Edge Blocks
    - "Armrest" Blocks
    - "Backrest" Blocks
  - Corners
    - "Leg" Blocks
  - Middle "Cushion" Blocks

GRID STAGE

The base grid was generated with a simple grid generation algorithm. The max length (l_{max}) and max width (w_{max}) of the base is user controlled, and based on that, the options from which the algorithm can generate a $\{x, y\}$ grid, where there can exist any number of combinations for $x(min = 2, max = l_{max})$ and $y(min = 2, max = w_{max})$. The grid has base alternating colors between green and grey to initially (to indicate the "cushion" part of a seat).

Once the grid is generated, based on a modulus algorithm, the edges are determined, with respect to its $\{x, y\}$ position on the grid. Once the edges are determined, the algorithm marks them as "Edges" from a list of enumerators, and recolors them as red, and adds them to a list of current edges.

Further, extending the edge detection logic, the corners are determined, marked as "Corners" from the enumerator, added to a list, and then recolored yellow. The corner list entries are removed from the initial edge list, for design purposes and elimination of redundancy.

This sets up the base, and the first step is complete.

APPEND STAGE

In this next stage, the two edge and corner lists are used to add blocks that represent "legs" and "backs" of the seat. The algorithm takes into account the starting block (be it an edge or corner) and based on its enumerator type, either instantiate back blocks on the positive Y-axis or leg blocks on the negative Y-axis. Once the required number of leg blocks and back blocks are instantiated, all these along with the grid blocks are moved under a single parent object, to preserve hierarchy and control.

CLEANUP STAGE

When the button for generation is pressed, then the edge and corner lists are cleared, thus making way for newer variants. A new set of grid blocks, leg blocks, and back blocks are generated, and all the three stages repeat.

However, an issue that can happen is when the blocks are removed from the list, they might host null values. These null values can cause errors in placement of the blocks. As a result, a cleanup algorithm runs in the background to clear all null values from the list.

Some images of the code and the two main scripts:

```
//Setting offset color for seat
var isOffset = (x % 2 == 0 && y % 2 != 0) || (x % 2 != 0 && y % 2 == 0);
spawnedBlock.Init(isOffset);

//Checking the edges
var isEdge = (x == 0 && y == 0) || (x % _length != 0 && y % (_width - 1) == 0) || (x % (_length - 1) == 0 && y % _width != 0);
spawnedBlock.CheckEdge(isEdge);
```

```
//Checking for corners
var checkCorner = (block_x_loc == 0 && block_y_loc == 0) ||
                  (block_x_loc % (ChairManager.chairManagerInstance.currentSeatLength - 1) == 0 &&
                  block_y_loc % (ChairManager.chairManagerInstance.currentSeatWidth - 1) == 0);
if(checkCorner)
{
    isCorner = true;
    currBlockType = BlockType.corner;
    meshRenderer.material.color = cornerColor;
    ChairManager.chairManagerInstance.cornerPosition.Add(this.gameObject.transform);
    ChairManager.chairManagerInstance.edgePosition.Remove(this.gameObject.transform);
}
```